When you write object-oriented programs, you're looking to model concepts from the outside world or from you logical domain. During this modeling process you'll discover categories of related data and behavior that need to be represented in code. In a system representing a jukebox, the concept of a "song" could be such a category. A song might combine state (for example, the name of the song) and methods that use that state (perhaps a method to play the song). In Ruby, you'd define a *class* called `Song` to represent the general case of what songs do.

Once you have these classes, you'll typically want to create a number of separate *instances* of each. For the jukebox system containing a class called `Song`, you'd have separate instances for popular hits with different names such as "Ruby Tuesday," "Enveloped in Python," "String of Pearls," "Small Talk," and so on. Each of these instances has their own state but shares the common behavior of the class. The word *object* is often used interchangeably with *instance*.

In Ruby, instances are created by calling a *constructor*, which is a special method associated with a class. The standard constructor is called `new`. As we'll see later in Chapter 3, Classes, Objects, and Variables, on page 33, the `new` method is defined for you by Ruby, and you don't need to define it on your own. You might create instances like this:

```
song1 = Song.new("Ruby Tuesday")
song2 = Song.new("Enveloped in Python")
# and so on
```

These instances are both derived from the same class, but they each have unique characteristics. Every object has a unique *object identifier* (abbreviated as *object ID*), accessible via the property `object_id`. In the above example, if you were to check `song1.object_id` and `song2.object_id`, you would find they have different values.

For each instance, you can define *instance variables*, variables with values that are unique to that instance. These instance variables hold an object's state. Each of our songs, for example, will have an instance variable that holds that song's title.

Within each class, you can define *instance methods*. Each method is a chunk of functionality that may be called in the context of the class and usually from outside the class, although you can set constraints on what methods can be used externally. These instance methods have access to the object's instance variables and hence to the object's state. A `Song` class, for example, might define an instance method called `play`. If a variable referenced a particular `Song` instance, you'd be able to call that instance's `play` method and play that song.

Syntactically, a method is invoked using dot syntax, here are some examples:

intro/puts_examples.rb

```
"gin joint".length  # => 9
"Rick".index("c")   # => 2
42.even?            # => true
sam.play(song)      # => "duh dum, da dum de dum ..."
```

Each line shows a method being called. The item before the period is called the *receiver* of the method, and what comes after the period is the name of the method to be invoked. The first example asks the string `"gin joint"` for its length; the second asks a different string to find the index within it of the letter *c*. The third line asks the number 42 if it is even (the question mark is part of the method name `even?`). Finally, we ask an object called `sam` to play us a song